

DEAP 2: Mixed-Variable Optim



DISTRIBUTED
EVOLUTIONARY
ALGORITHMS IN
PYTHON

Developed by a team at Université Laval in Quebec, Canada. The key contributors include Francois-Michel De Rainville, Marc-André Gardner, Christian Gagné, and Marc Parizeau.

LTU CS CJ Chung

So far variables were continuous

```
# Define bounds for each variable as (min, max)
BOUNDS_LOW = [-10, -5, -2]    # Minimum bounds for variables
BOUNDS_HIGH = [10, 5, 10]     # Maximum bounds for variables

# Create a fitness and individual class
creator.create("FitnessMin", base.Fitness, weights=(-1.0,)) # Minimize objective
creator.create("Individual", list, fitness=creator.FitnessMin)

# Initialize DEAP toolbox
toolbox = base.Toolbox()

# Attribute generator that initializes within the variable bounds
def create_individual():
    return [random.uniform(low, high) for low, high in zip(BOUNDS_LOW, BOUNDS_HIGH)]

# Register individual and population creation functions
toolbox.register("individual", tools.initIterate, creator.Individual, create_individual)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
```

Mixed-Variable Optimization

DEAP_MixedVarOp.ipynb on Canvas

- **Variable 1:** Continuous, bounded between [-5, 5].
- **Variable 2:** Continuous, bounded between [0, 5].
- **Variable 3:** Discrete, can only take values from the set {1, 2, 3}.
- **Objective function:**

$$f(x_1, x_2, x_3) = (x_1 - 4)^2 + (x_2 - 2.35)^2 + (x_3 - 2)^2$$

```
LOW = [-5, 0, 1]
```

```
HIGH = [5, 5, 3]
```

A simple solution using built-ins (Multiple Methods are possible)

- treating all variables as continuous and then converting discrete variables appropriately.

DEAP_MixedVarOp.ipynb

```
# Define the optimization problem as minimization
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)

LOW = [-5, 0, 1] # Minimum bounds for variables
HIGH = [ 5, 5, 3] # Maximum bounds for variables
N_VAR = len(LOW)

# Define the toolbox
toolbox = base.Toolbox()
# Variable 1: Continuous variable
toolbox.register("attr_float1", random.uniform, LOW[0], HIGH[0])

# Variable 2: Continuous variable
toolbox.register("attr_float2", random.uniform, LOW[1], HIGH[1])

# Variable 3: Discrete variable from the set {1, 2, 3}
toolbox.register("attr_int", random.choice, range(LOW[2]-1, HIGH[2], 1)) # (start, stop, step)
# range(3, 6) ==> [3,4,5] step is 1 by default

# Combine variables into an individual
toolbox.register("individual", tools.initCycle, creator.Individual,
                 (toolbox.attr_float1, toolbox.attr_float2, toolbox.attr_int), n=1)

# Define the population
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
```

DEAP_MixedVarOp.ipynb

```
def eval_function(individual): # Define the objective function
    x1, x2, x3 = individual
    return ((x1 - 4) ** 2 + (x2 - 2.35) ** 2 + (x3 - 2) ** 2,)

toolbox.register("evaluate", eval_function)

# Define genetic operators
toolbox.register("mate", tools.cxBlend, alpha=0.5) # Blend crossover for continuous variables
toolbox.register("mutate_cont", tools.mutGaussian, mu=0, sigma=2, indpb=0.5) # Continuous mutation
toolbox.register("mutate_int", tools.mutUniformInt, low=LOW[2], up=HIGH[2], indpb=1.0) # Discrete mutation
toolbox.register("select", tools.selTournament, tournsize=3)

# Custom mutation to handle both continuous and discrete parts
def custom_mutate(individual):
    # Mutate continuous variables (x1 and x2)
    toolbox.mutate_cont(individual[:N_VAR-1])
    # Mutate discrete variable (x3)
    toolbox.mutate_int(individual[N_VAR-1:])
    return individual,

def check_bounds(individual):
    individual[0] = np.clip(individual[0], LOW[0], HIGH[0])
    individual[1] = np.clip(individual[1], LOW[1], HIGH[1])
    individual[2] = np.clip(individual[2], LOW[2], HIGH[2])
    return individual, 0

toolbox.register("mutate", custom_mutate)
toolbox.decorate("mutate", tools.DeltaPenalty(check_bounds, 0)) # Use check_bounds function
```

```
# Main algorithm
```

```
POP_SZ = 35
```

```
def main():
```

```
    random.seed(77)
```

```
    # Create a population
```

```
    population = toolbox.population(n=POP_SZ)
```

```
    # Statistics to track
```

```
    stats = tools.Statistics(lambda ind: ind.fitness.values)
```

```
    stats.register("min", min)
```

```
    # Extract the fitness value from the tuple for calculating average
```

```
    stats.register("avg", lambda lst: sum(x[0] for x in lst) / len(lst))
```

```
    # Run the genetic algorithm
```

```
    population, logbook = algorithms.eaSimple(population, toolbox,
```

```
        cxpb=0.7, mutpb=0.2,
```

```
        ngen=20, stats=stats, verbose=True)
```

```
    return population, logbook
```

```
if __name__ == "__main__":  
    final_population, log = main()
```

```
    # Display the best solution
```

```
    best_individual = tools.selBest(final_population, k=1)[0]
```

```
    print("Best Individual:", best_individual)
```

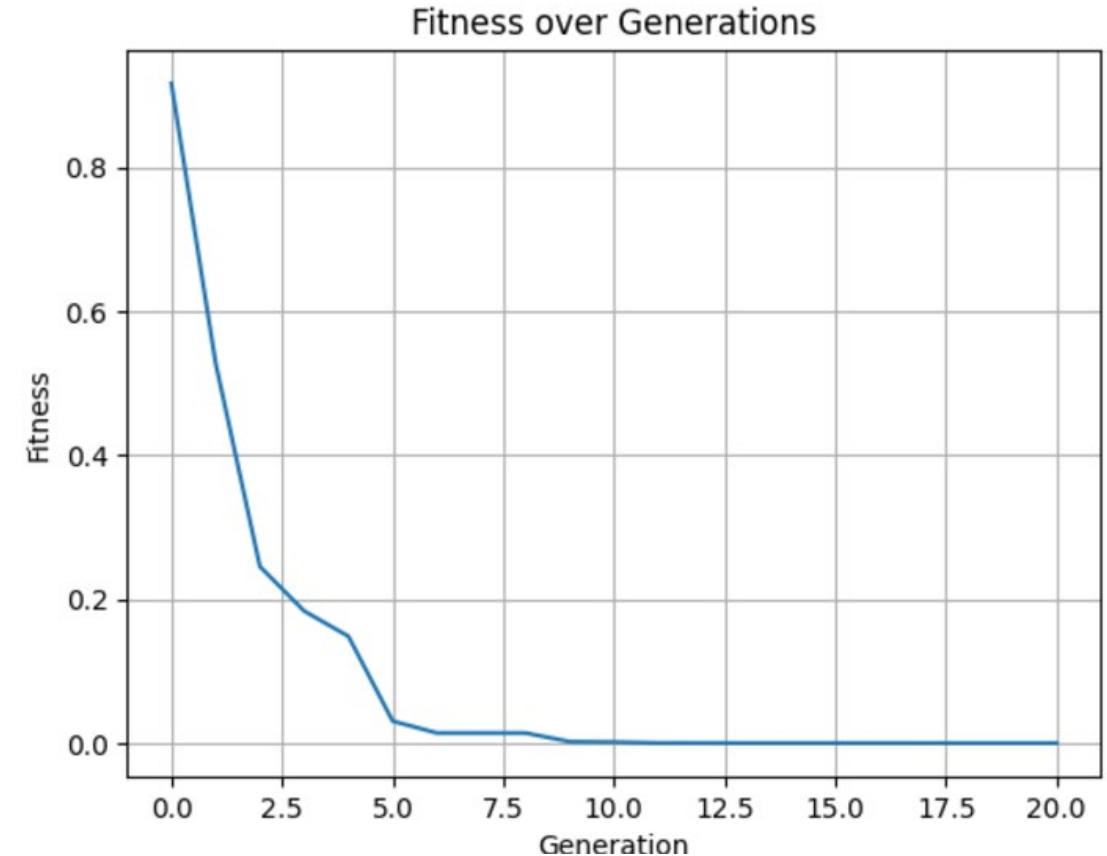
```
    print("Fitness:", best_individual.fitness.values)
```

```
    print(f"Final Population with {POP_SZ} individuals - {final_population}")
```

gen	nevals	min	avg
0	35	(0.9168292515984304,)	26.4
1	28	(0.5278228682824884,)	5.37701
2	27	(0.2453928872939402,)	2.59263
3	25	(0.18353394523286282,)	1.54247
4	28	(0.1484251621006669,)	0.75411
5	26	(0.030564821076103556,)	0.439344
6	26	(0.014087939500794877,)	0.240363
7	25	(0.014087939500794877,)	0.163614
8	22	(0.014087939500794877,)	0.110474
9	22	(0.00202660083446667,)	0.0504244
10	22	(0.0014876606316188057,)	0.0229636
11	28	(0.00018807869661892275,)	0.00925237
12	23	(0.0001497577981329251,)	0.00182168
13	25	(6.364220508380118e-05,)	0.000559671
14	27	(5.694301980523731e-05,)	0.00022419
15	28	(5.694301980523731e-05,)	0.000123142
16	24	(5.65848442343538e-05,)	8.28572e-05
17	23	(4.5099745201654016e-05,)	6.89701e-05
18	20	(4.5099745201654016e-05,)	5.55987e-05
19	20	(4.444610553872471e-05,)	5.22458e-05
20	22	(3.915992133032458e-05,)	4.8052e-05

Best Individual: [3.999503719473601, 2.350379884325303, 2.0062265010133165]
 Fitness: (3.915992133032458e-05,)

Population size used: 35

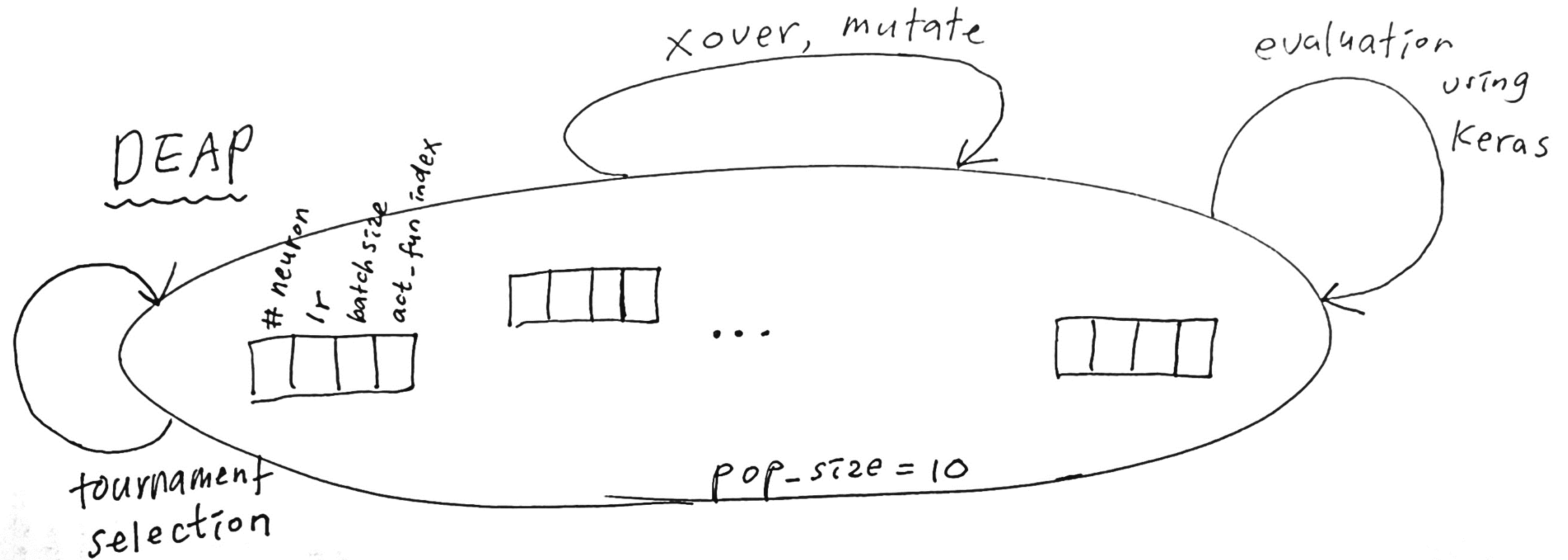


HPO for a func regression problem using Keras

DEAP_HPO_-x^2.ipynb on Canvas

- **Variable 0 for # neurons:** Continuous, bounded between [4, 64].
- **Variable 1 for lr:** Continuous, bounded between [0, 5].
- **Variable 2 for batch size:** Continuous, bounded between [0, 5].
- **Variable 3 for activation function index:** Discrete, can only take values from the set {0, 1}.
- **Objective function:** Keras model to return loss. See next slide

```
LOW = [4, 0.001, 4, 0]  
HIGH = [64, 1.5, 10, 1]
```



```
# Use (n, 1) shape instead of (n,), since (n, 1) is general input format for Keras DNN models
X = np.linspace(0, 1, 30).reshape(-1, 1) # -1 simply means that it is an unknown dimension and
y = -X*X                                # we want numpy to figure it out.
Xt = np.linspace(0, 1, 7).reshape(-1, 1)
yt = -Xt * Xt
```

DEAP_HPO_-x^2.ipynb

```
# num_neurons, lr, bsize, act_func
def eval_function(param):
    global m # to plot
    m = Sequential()
    m.add(Input(shape=(1,)))
    if int(param[3]) == 0:
        m.add(Dense(abs(int(param[0])), activation='relu'))
    elif int(param[3]) == 1:
        m.add(Dense(abs(int(param[0])), activation='sigmoid'))
    m.add(Dense(1))

    m.compile(optimizer=optimizers.SGD(learning_rate=param[1]), loss='mse')
    cb_list = [
        callbacks.EarlyStopping(monitor='val_loss', patience=5,
                                restore_best_weights=True),
    ]
    m.fit(X, y, epochs=300, batch_size=int(param[2]), verbose = 0,
          # validation_split=0.2, callbacks=cb_list) <== may not work since split just cuts the last part
          validation_data=(Xt, yt), callbacks=cb_list)
    loss = m.evaluate(Xt, yt, verbose=0)
    return (loss,)
```

```

toolbox.register("evaluate", eval_function)

# Define genetic operators
toolbox.register("mate", tools.cxBlend, alpha=0.5) # Blend crossover for continuous variables
toolbox.register("mutate_cont", tools.mutGaussian, mu=0, sigma=2, indpb=0.5) # Continuous mutation
toolbox.register("mutate_int", tools.mutUniformInt, low=LOW[3], up=HIGH[3], indpb=0.2) # Discrete mutation
toolbox.register("select", tools.selTournament, tournsize=3)

# Custom mutation to handle both continuous and discrete parts
def custom_mutate(individual):
    # Mutate continuous variables (x0, x1 and x2)
    toolbox.mutate_cont(individual[:N_VAR-1])

    # Mutate discrete variable (x3)
    toolbox.mutate_int(individual[N_VAR-1:])
    return individual,

def check_bounds(individual):
    individual[0] = abs(np.clip(individual[0], LOW[0], HIGH[0])) # make sure positive ... ??
    individual[1] = abs(np.clip(individual[1], LOW[1], HIGH[1]))
    individual[2] = abs(np.clip(individual[2], LOW[2], HIGH[2]))
    individual[3] = abs(np.clip(individual[3], LOW[3], HIGH[3]))
    return individual, 0

toolbox.register("mutate", custom_mutate)
toolbox.decorate("mutate", tools.DeltaPenalty(check_bounds, 0)) # Use check_bounds function

```

```
if __name__ == "__main__":  
    final_population, log = main()
```

```
# Main algorithm
```

```
def main():
```

```
    random.seed(77)
```

```
    # Create a population
```

```
    population = toolbox.population(n=10)
```

```
    # Statistics to track
```

```
    stats = tools.Statistics(lambda ind: ind.fitness.values)
```

```
    stats.register("min", min)
```

```
    # Extract the fitness value from the tuple for calculating average
```

```
    stats.register("avg", lambda lst: sum(x[0] for x in lst) / len(lst))
```

```
    # Run the genetic algorithm
```

```
    population, logbook = algorithms.eaSimple(population, toolbox,  
                                              cxbp=0.5, mutpb=0.3,  
                                              ngen=5, stats=stats, verbose=True)
```

```
    return population, logbook
```

```
    # Display the best solution
```

```
    best_individual = tools.selBest(final_population, k=1)[0]
```

```
    print("Best Individual:", best_individual)
```

```
    print("Fitness:", best_individual.fitness.values)
```

```
    print(final_population)
```

gen	nevals	min	avg
0	10	(0.0003076366556342691,)	62589.1
1	8	(0.0002525205200072378,)	257.466
2	6	(0.00021630885021295398,)	0.67916
3	5	(0.00021630885021295398,)	0.000742307
4	7	(0.0002525205200072378,)	0.000556025
5	8	(0.00012635944585781544,)	0.000394388

Best Individual: [16.298576634644718, 0.28834021401966253, 4.485693820360006, 0.0]

Fitness: (0.00012635944585781544,)

